

## DEEP REINFORCEMENT LEARNING

# Monte Carlo Methods

Estimating values from sampled episodes

Companion reader · Read beside the slides

**How to use this companion.** Colour boxes guide you: **blue** intuition, **amber** everyday picture, **green** worked example, **red** watch out, **purple** key takeaway. Read it next to the slides, in order.

## 1 Where Monte Carlo fits in

So far we solved control problems with **dynamic programming** (DP): given the transition probabilities  $p(s', r | s, a)$  and rewards, we computed values exactly by bootstrapping — each value was built from *other* estimated values. That is powerful but it has one fatal catch: **you need a model of the world**. In most real problems (a robot on a track, a game of blackjack, a trading book) we do *not* have that model. **Monte Carlo** (MC) methods solve exactly this case: they learn values purely from **experience** — sampled episodes of states, actions and rewards — with *no model* and *no bootstrapping*.

The name comes from the casino: a **Monte Carlo method** is any algorithm that estimates a quantity by *repeated random sampling*. The trick is always the same — when a quantity is hard to compute analytically, draw many unbiased samples and average them. The same idea estimates  $\pi$  by throwing darts, prices options in finance, and renders light in computer graphics. In RL the “hard quantity” is the value of a state, and the “samples” are the returns we actually receive when we run the policy.

### The intuition

“Play the game to the end, many times, and average what you got.” To value a position, MC plays out full episodes starting from it and takes the mean return — exactly like estimating a coin’s bias by flipping it a few hundred times instead of reasoning about its physics.

### Everyday picture

A new cafe owner wants to know how good Table 7 is. She has no formula for “table goodness”. So she just records the total tip every time a customer is seated there, over a few hundred visits, and averages. That average *is* the Monte Carlo value of Table 7 — learned from outcomes, not from a model of customers.

MC works on **episodic** tasks: tasks that always reach a terminal state (a game ends, the car finishes or crashes, the hand is played out). We run the policy to termination, observe the total discounted reward, and use it. There is an offline/online distinction worth naming: classic MC is naturally an **offline** (batch) method — it waits for an episode to *finish* before it can update, because the return is only known at the end. (Temporal-difference learning, next session,

removes that wait by bootstrapping.)

## 2 The return, and the goal of prediction

Everything rests on one definition. The **return** from time  $t$  is the total discounted reward that actually followed:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T.$$

The value of a state is the *expected* return under the policy,  $V_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ . An expectation is just an average over infinitely many samples — so if we collect many actual returns from  $s$  and average them, the **law of large numbers** guarantees the average converges to  $V_\pi(s)$ . That single sentence is the whole of MC prediction.

## 3 First-visit Monte Carlo prediction

To estimate  $V_\pi(s)$  for every state, run many episodes under  $\pi$  and, for each episode, walk *backwards* accumulating the return  $G$ . The classic algorithm is:

1. Initialise  $V(s)$  arbitrarily and an empty list  $Returns(s)$  for every state.
2. Generate an episode  $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$  following  $\pi$ .
3. Set  $G \leftarrow 0$ . For  $t = T-1, T-2, \dots, 0$ : set  $G \leftarrow \gamma G + R_{t+1}$ .
4. If  $S_t$  does *not* appear in  $S_0, \dots, S_{t-1}$  (**first visit**), append  $G$  to  $Returns(S_t)$  and set  $V(S_t) \leftarrow \text{average}(Returns(S_t))$ .

**First-visit MC** averages the return after the *first* time a state is reached in an episode; **every-visit MC** averages the return after *every* occurrence. They are subtly different estimators (first-visit returns are independent, so its theory is cleaner) but both converge to  $V_\pi(s)$  as the number of visits grows.

### Worked example: first-visit MC on the Berkeley 5-state gridworld ( $\gamma = 1$ )

States A, B, C, D, E. Every move scores  $-1$ ; exiting at D gives  $+10$ , exiting at A gives  $-10$ . We observe four episodes under a fixed policy and read off the return to each *first-visited* state (with  $\gamma = 1$  the return is just “rewards from here to the end”):

| Episode (state, action, reward) | First-visit returns    |
|---------------------------------|------------------------|
| 1: B,e,-1; C,e,-1; D,exit,+10   | B= 8, C= 9, D= 10      |
| 2: B,e,-1; C,e,-1; D,exit,+10   | B= 8, C= 9, D= 10      |
| 3: E,n,-1; C,e,-1; A,exit,-10   | E= -12, C= -11, A= -10 |
| 4: E,n,-1; C,e,-1; D,exit,+10   | E= 8, C= 9, D= 10      |

Averaging each state’s first-visit returns:

$$V(A) = -10, \quad V(B) = 8, \quad V(C) = \frac{9+9-11+9}{4} = 4, \quad V(D) = 10, \quad V(E) = \frac{-12+8}{2} = -2.$$

Notice C: it sits on the path to *both* exits, so its return is sometimes  $+9$  (heading to D) and once  $-11$  (heading to A); these average to  $+4$ . State A was only ever the terminal-bound state, so  $V(A) = -10$ .

**⚠ Watch out**

Look at  $B = +8$  and  $E = -2$ , yet *both* states transition into  $C$ . How can two states that lead to the same next state have such different values? Because direct MC **ignores the model**: it does not know  $B$  and  $E$  share a successor, so it learns each state in isolation from whatever episodes happened to pass through it. The asymmetry is an artefact of *which* episodes were sampled ( $B$ 's episodes went to the  $+10$  exit; one of  $E$ 's went to the  $-10$  exit), not of the dynamics. DP, which uses the model, would couple them; MC will agree only in the limit of infinite data.

## 4 What direct MC evaluation gets right — and wrong

**Strengths.** It is conceptually trivial (“average the returns”); it needs *zero* knowledge of the model; the estimates for one state do not depend on the accuracy of estimates for any other state (no bootstrapping means no propagated bias); and it provably converges to the true values. It is also robust to violations of the Markov property, because it never relies on one-step structure.

**Weaknesses.** It *wastes* the information contained in transitions — the fact that  $B$  and  $E$  both feed  $C$  is thrown away, so shared structure is re-learned from scratch. Each state must be visited enough times *on its own*, so learning is slow and sample-hungry. And because an update needs a complete return, you cannot learn until each episode *ends* (no good for very long or non-terminating tasks).

## 5 Incremental Monte Carlo

Storing every return is wasteful. The running mean of a stream can be updated in place. If  $V_n$  is the average of the first  $n$  returns and  $G$  is the next one, then

$$V_{n+1} = V_n + \frac{1}{n+1}(G - V_n).$$

This is the same “**new estimate**  $\leftarrow$  **old estimate** + step  $\times$  (target – old)” rule we first met for multi-armed bandits — now the “target” is the episode return  $G$  and the step size is  $1/N(s)$ . Writing it for state values:

$$V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)}(G_t - V(S_t)).$$

With  $1/N$  the step shrinks over time and we recover the exact sample average (good for a *stationary* problem). Replacing it with a *constant* step  $\alpha$  gives an exponentially-weighted recency average,  $V \leftarrow V + \alpha(G - V)$ , which keeps adapting — the right choice when the environment or policy is **non-stationary** and old returns should fade.

## 6 The exploration problem and Monte Carlo control

Prediction values a fixed policy; **control** improves it. MC control follows **generalised policy iteration** (GPI): alternately *evaluate* the current policy with MC, then *improve* it greedily. But here a model-free wrinkle bites. To improve  $V$  greedily you would need to look one step ahead through  $p(s', r \mid s, a)$  — which we do not have. The fix is to estimate **action values**  $Q(s, a)$  directly: once you know  $Q$ , acting greedily is immediate,  $\pi(s) = \arg \max_a Q(s, a)$ , no model required.

**⚠ Watch out**

A purely greedy policy is deterministic, so for each state it only ever takes *one* action — the other actions'  $Q(s, a)$  are never sampled and stay unknown forever. You cannot find the best action if you never try it. Two remedies guarantee every action is explored:

- **Exploring starts:** begin episodes from a random  $(s, a)$  pair so that, in the limit, every state-action pair starts infinitely many episodes.
- **$\epsilon$ -soft /  $\epsilon$ -greedy policies:** keep every action's probability  $\geq \epsilon/|A|$ , so exploration is baked into the behaviour itself and exploring starts are not needed.

**MC control with exploring starts** interleaves the two halves of GPI per episode: generate an episode from a random  $(s, a)$ , update  $Q$  for first-visited pairs by averaging returns, then make  $\pi$  greedy w.r.t. the new  $Q$ . Because both evaluation and improvement move toward the optimum, the loop converges to  $Q_*$  and  $\pi_*$ .

## 7 On-policy vs off-policy learning

A control method is **on-policy** if it evaluates and improves the *same* policy that is generating the data. On-policy MC uses an  $\epsilon$ -soft policy and converges to the best policy *within the  $\epsilon$ -soft family* — it can never be fully greedy, because it must keep exploring. That is the price of learning from your own behaviour: a residual amount of randomness you can never switch off.

**Off-policy** learning breaks that compromise. It uses *two* policies: a **target** policy  $\pi$  we want to evaluate/optimize (can be fully greedy), and a **behaviour** policy  $b$  we actually follow to generate data (can be exploratory). The one requirement is **coverage**:  $b(a | s) > 0$  wherever  $\pi(a | s) > 0$  — the behaviour must be *able* to take any action the target might. This separates “learning the best thing” from “trying enough things”.

## 8 Importance sampling: reusing $b$ 's data to value $\pi$

There is a catch: returns observed under  $b$  are samples of  $G$  under  $b$ 's distribution, not  $\pi$ 's. We must re-weight them. The probability of a trajectory after  $S_t$  under any policy factorises over its action probabilities (the environment's  $p$  terms are *shared* by both policies), so their ratio — the **importance-sampling ratio** — depends only on the policies:

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}.$$

Weighting each behaviour return by  $\rho$  turns it into an unbiased sample of the target return, because  $\mathbb{E}_b[\rho_{t:T-1} G_t | S_t = s] = V_\pi(s)$ . Two ways to average:

- **Ordinary** importance sampling:  $V(s) = \frac{\sum \rho_{t:T-1} G_t}{N(s)}$  — *unbiased*, but the variance can be enormous (even *infinite*) because a single large  $\rho$  swamps the average.
- **Weighted** importance sampling:  $V(s) = \frac{\sum \rho_{t:T-1} G_t}{\sum \rho_{t:T-1}}$  — normalise by the sum of the ratios instead of the count. It is *biased* (the bias vanishes as data grows) but has much lower, *bounded* variance, and is strongly preferred in practice.

 **Worked example: ordinary vs weighted on a blackjack state**

Target policy: “stick on 19 or below” (deterministic). Behaviour policy: hit/stick uniformly at random. Both estimators converge to the true value of the state, but if you plot estimate vs. number of episodes, **ordinary** IS swings wildly for the first few thousand episodes (a rare trajectory with a huge  $\rho$  yanks the mean), while **weighted** IS sits close to the truth almost immediately. Same data, same limit — but weighted IS is far more useful early, which is when you actually care.

## 9 Putting it together: off-policy MC control

Combine the pieces and you get off-policy MC control. Walking each episode backwards, maintain a cumulative weight  $W$  (the running product of  $\pi/b$ ), update each first-visited  $Q(S_t, A_t)$  with a *weighted* incremental rule

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} (G - Q(S_t, A_t)),$$

where  $C$  accumulates the weights, then set  $\pi(S_t) = \arg \max_a Q(S_t, a)$ . If at any step the action taken was *not* the one the greedy target would pick,  $\pi(A_t | S_t) = 0$ , so  $W \rightarrow 0$  and we stop walking back that episode (the rest carries no information about the greedy target). This learns an optimal greedy policy while behaving exploratorily throughout.

 **Key takeaway**

Monte Carlo learns values by *averaging actual returns over complete episodes* — model-free and bootstrap-free. Use first-/every-visit averaging for **prediction**; estimate  $Q$  and run GPI with **exploring starts** or an  $\epsilon$ -**soft** policy for **control**; and re-weight with **importance sampling** (weighted < ordinary for variance) for **off-policy** learning. Versus DP, MC needs no model; versus TD (next), MC is unbiased but higher-variance and must wait for episodes to end.